ELE 216 Laboratoire 6 : Application intégrée — Banc de test

Ce laboratoire est la suite directe du laboratoire 5. Vous intégrez les modules `resultat`, `journal` et `sequenceur` dans une application de banc de test pilotée par un interpréteur de commandes textuelles, capable de charger et d'exécuter des fichiers de test.

Prérequis : vos modules du laboratoire 5 doivent compiler et passer leurs tests. Placez-les dans un répertoire `../labo5` (ou copiez les fichiers nécessaires) et ajoutez le chemin dans `INCLUDES`.

Vous devez réaliser ce laboratoire seul, pendant la période. Il vous est permis de discuter avec vos collègues, consulter les chargés de laboratoires et l'internet.

Vous devez faire valider votre travail avant de quitter. On vous demandera de charger et exécuter un fichier de test devant le chargé de laboratoire, puis d'expliquer le fonctionnement de votre interpréteur.

Problème 1 : Interpréteur de commandes

Le module banc intègre le journal et le séquenceur du laboratoire 5 derrière un interpréteur de commandes textuelles. Chaque ligne contient une commande suivie de ses paramètres, séparés par des espaces. Les lignes vides et les lignes commençant par # sont ignorées.

Exemple de session :

```
set voltage 3.3
set current 0.5
set tolerance 5
push
set voltage 5.0
measure vregulator_5v 5.0
pop
measure vregulator_3v3 3.3
report
save results.txt
quit
```

Interface :

`banc_t *banc_creer(void);` alloue le banc, son journal et son séquenceur.

`void banc_detruire(banc_t *b);` libère le banc et tous ses sous-modules.

`int banc_interpreter_ligne(banc_t *b, const char *ligne);` analyse une ligne avec strtok, construit la commande, l'enfile et l'exécute immédiatement. Retourne 1 si la commande a été traitée, 0 si la ligne est vide ou un commentaire, -1 si la commande est inconnue.

`int banc_charger_fichier(banc_t *b, const char *chemin);` ouvre un fichier, enfile toutes les commandes, puis les exécute séquentiellement. Retourne 1 si le fichier a été traité, 0 si le fichier est introuvable.

`void banc_boucle_interactive(banc_t *b);` affiche un prompt, lit des lignes sur stdin avec fgets, appelle banc_interpreter_ligne en boucle jusqu'à la commande quit.

`void banc_afficher_etat(const banc_t *b);` affiche le contexte courant du séquenceur et le résumé du journal.

Commandes reconnues :

`set voltage <V>` — fixe la tension d'alimentation du contexte.

`set current <A>` — fixe le courant limite du contexte.

`set tolerance <%>` — fixe la tolérance par défaut du contexte.

`push` — sauvegarde le contexte courant.

`pop` — restaure le dernier contexte sauvegardé.

`measure <id> <valeur_attendue>` — simule une mesure et l'enregistre dans le journal.

`report` — affiche le rapport complet des résultats.

`save <chemin>` — sauvegarde le rapport dans un fichier texte.

`load <chemin>` — charge et exécute un fichier de commandes.

`status` — affiche l'état courant du banc (contexte + résumé journal).

`quit` — termine la boucle interactive.

Gestion des erreurs :

- commande inconnue : afficher un message d'erreur, ne pas planter
- fichier introuvable : afficher un message, retourner au prompt
- paramètres manquants : ignorer la commande avec un avertissement

Tests unitaires :

- création et destruction du banc sans fuite
- parsing correct de chaque type de commande (set, push, pop, measure, report, save, load, status, quit)
- commande inconnue retourne -1
- ligne vide et commentaire (#) retournent 0
- exécution d'une séquence connue, vérification du nombre de résultats dans le journal (fixer le seed avec `srand(42)` pour reproductibilité)

Problème 2 : Fichiers de test

Écrivez trois fichiers de commandes qui exercent différents aspects du banc de test. Chaque fichier doit se terminer par un rapport. Chargez-les et exécutez-les depuis votre programme principal.

Fichier 1 : test_alimentation.txt

Test d'une alimentation double sortie 3.3V / 5V. Utilise push/pop pour changer de configuration entre les deux tensions.

```
# Test alimentation double sortie
set voltage 3.3
set current 0.5
set tolerance 5

measure rail_3v3 3.3
measure ripple_3v3 0.02

push
set voltage 5.0
set tolerance 8
measure rail_5v 5.0
measure ripple_5v 0.04
pop

measure rail_3v3_post 3.3
report
save resultats_alim.txt
```

Fichier 2 : test_capteurs.txt

Test d'un ensemble de capteurs analogiques avec une tolérance serrée (2 %).

```
# Test de capteurs analogiques
set voltage 3.3
set tolerance 2

measure temp_sensor 0.75
measure pressure_sensor 1.50
measure humidity_sensor 2.20
measure light_sensor 0.33

report
save resultats_capteurs.txt
```

Fichier 3 : test_stress.txt

Test de stress avec configurations multiples et push/pop imbriqués. Ce fichier exerce le comportement du séquenceur avec plusieurs niveaux de contexte.

```
# Test de stress : push/pop imbriqués
set voltage 3.3
set tolerance 5
measure base_3v3 3.3

push
```

```
set voltage 5.0
set tolerance 10
measure base_5v 5.0

  push
  set voltage 12.0
  set tolerance 15
  measure base_12v 12.0
  pop

# Retour à 5V
measure retour_5v 5.0
pop

# Retour à 3.3V
measure retour_3v3 3.3
status
report
save resultats_stress.txt
```

Programme principal

Le main orchestre la démonstration :

crée un banc de test
charge et exécute les trois fichiers de test
affiche l’état final du banc
entre en mode interactif pour permettre des tests supplémentaires au clavier
libère toute la mémoire après quit